PROJET SAE S4 – 2026

Emoji Slither

BATTLE ROYALE EDITION

Organisation & Planification du Projet

Document de gestion de projet – Semestre 4

Yani NAIT CHABANE	Amine KSANTINI	Malcolm MONTCHO
Backend & Game Engine	Frontend & Rendu Canvas	Interface Mobile & UX

Groupe 4E – BUT 2 Informatique
IUT d'Orsay – Université Paris-Saclay

Mars 2026

Table des matières **2**
1. Introduction **3**
2. Liste Détaillée des Tâches **3**
3. Diagramme de PERT **6**
3.1 Représentation du graphe PERT 6
4. Diagramme de GANTT **7**
4.1 Jalons du projet 7
5. Répartition des Tâches **9**
5.1 Yani NAIT CHABANE – Backend & Game Engine 9
5.2 Amine KSANTINI – Frontend & Rendu Canvas 9
5.3 Malcolm MONTCHO – Interface Mobile & UX 9
5.4 Tâches communes (Tous) 10
6. Organisation de l'Équipe **11**
6.1 Outils de Communication 11
6.2 Versioning et Gestion du Code 11
6.3 Environnements de Développement 11
6.4 Méthode de Travail 12
6.5 Gestion des Risques 12
7. Conclusion **12**

1. Introduction

Ce document présente le plan d'organisation et de planification du projet Emoji Slither Battle Royale, développé dans le cadre de la SAE S4 du BUT 2 Informatique à l'IUT d'Orsay. Il a pour objectif de définir précisément comment l'équipe va s'organiser pour mener à bien ce projet en 6 semaines.

Le projet consiste à développer un jeu massivement multijoueur colocalisé de type Battle Royale, inspiré de Slither.io, où les joueurs contrôlent leurs serpents via leur smartphone. Ce document détaille la liste exhaustive des tâches, les diagrammes de planification (PERT et GANTT), la répartition des responsabilités, ainsi que les outils et méthodes de travail de l'équipe.

2. Liste Détaillée des Tâches

Le projet est divisé en 6 grandes phases, alignées sur les 6 semaines de réalisation. Chaque tâche est accompagnée d'une estimation de durée réaliste tenant compte de la charge de formation et d'apprentissage.

Légende				
Yani	**Amine**	**Malcolm**	**Tous**	Tâches communes / Documentation

T1 – Setup & Architecture

ID	Sous-tâche	Responsable	Durée est.	Semaine
T1.1	Installation Node.js, VS Code, Git, npm	Tous	2h	S1
T1.2	Configuration dépôt GitHub (branches, issues)	Tous	2h	S1
T1.3	Initialisation projet Node.js + Express.js	Yani	2h	S1
T1.4	Intégration Socket.IO de base (serveur ↔ client)	Yani	3h	S1
T1.5	Création Canvas HTML5 et structure de la map	Amine	3h	S1
T1.6	Rédaction Game Design Document (GDD)	Tous	6h	S1

T2 – Prototype de base (serpent solo)

ID	Sous-tâche	Responsable	Durée est.	Semaine
T2.1	Logique serpent côté serveur (position, direction)	Yani	4h	S1-S2
T2.2	Intégration nipplejs (joystick virtuel mobile)	Malcolm	3h	S1-S2
T2.3	Rendu Canvas du serpent (tête + segments)	Amine	4h	S1-S2
T2.4	Système d'orbes (génération, collecte, réapparition)	Yani	4h	S1-S2
T2.5	Système de croissance par collecte d'orbe	Yani	3h	S1-S2
T2.6	Connexion Socket.IO mobile → serveur (player:move)	Malcolm	3h	S1-S2

T3 – Multijoueur & Collisions

ID	Sous-tâche	Responsable	Durée est.	Semaine
T3.1	Gestion multi-clients (30+ connexions simultanées)	Yani	5h	S2-S3
T3.2	Détection collision tête/corps (serpent A → corps B)	Yani	5h	S2-S3
T3.3	Détection collision tête/tête (double élimination)	Yani	3h	S2-S3
T3.4	Détection collision bordures de la map	Yani	2h	S2-S3
T3.5	Système d'élimination (mode spectateur, relâche orbes)	Yani	4h	S2-S3
T3.6	Rendu multi-serpents sur Canvas	Amine	4h	S2-S3
T3.7	Adaptation responsive iOS/Android	Malcolm	4h	S2-S3

T4 – Système Pièces / Coffres / Bonus

ID	Sous-tâche	Responsable	Durée est.	Semaine
T4.1	Génération et collecte des pièces d'or	Yani	3h	S3-S4
T4.2	Système de coffres (Bronze/Argent/Or, coûts)	Yani	4h	S3-S4

T4.3	Bonus Aimant (attraction orbes dans rayon 150px)	Yani	3h	S3-S4
T4.4	Bonus Invincibilité (immunité totale temporaire)	Yani	2h	S3-S4
T4.5	Bonus Fantôme (passage à travers les corps)	Yani	3h	S3-S4
T4.6	UI bonus sur manette mobile (bouton ACTIVER + timer)	Malcolm	3h	S3-S4
T4.7	Auras visuelles selon bonus actif	Amine	3h	S3-S4

T5 – Interface & Polish Visuel

ID	Sous-tâche	Responsable	Durée est.	Semaine
T5.1	Classement Top 5 temps réel (écran principal)	Amine	3h	S4-S5
T5.2	Kill Feed (notification éliminations)	Amine	2h	S4-S5
T5.3	Effets particules (explosion mort, collecte orbe)	Amine	5h	S4-S5
T5.4	Effets néon et trails sur serpents	Amine	3h	S4-S5
T5.5	Interface mobile responsive complète	Malcolm	4h	S4-S5
T5.6	Feedback haptique (vibrations mobile)	Malcolm	2h	S4-S5
T5.7	Écran de victoire avec stats finales	Amine	3h	S4-S5
T5.8	Countdown de démarrage de partie	Amine	1h	S4-S5

T6 – Tests, Finalisation & Documentation

ID	Sous-tâche	Responsable	Durée est.	Semaine
T6.1	Tests de charge (20-30 connexions simultanées)	Tous	4h	S6
T6.2	Optimisation performances serveur (60 ticks/sec)	Yani	4h	S6
T6.3	Débogage et corrections bugs	Tous	6h	S6
T6.4	Rédaction rapport Organisation & Planification	Tous	5h	S6
T6.5	Rédaction rapport final (technique + bilan)	Tous	6h	S6
T6.6	Préparation slides soutenance (15 min)	Tous	4h	S6
T6.7	Répétition soutenance + démonstration finale	Tous	3h	S6

Membre	Charge technique	Charge docs/tests	Total estimé
Yani NAIT CHABANE	~68h	~8h	**~76h**
Amine KSANTINI	~54h	~8h	**~62h**
Malcolm MONTCHO	~36h	~8h	**~44h**
Commun (Tous)	~14h	~24h	**~38h**
TOTAL PROJET	~172h	~48h	**~220h**

3. Diagramme de PERT

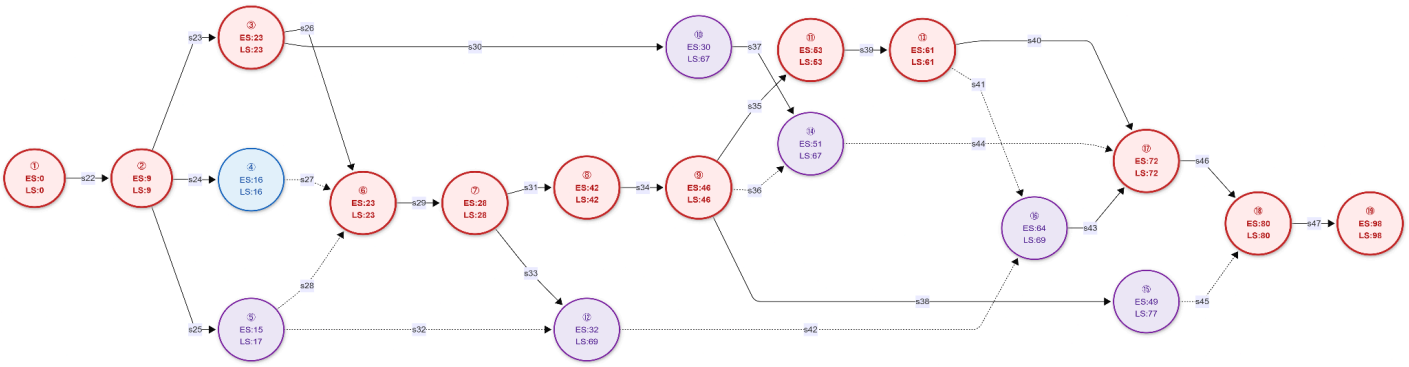
Le diagramme de PERT (Program Evaluation and Review Technique) permet de visualiser les dépendances entre les tâches et d'identifier le chemin critique du projet. Chaque tâche est identifiée par un code alphabétique, avec sa durée estimée et ses prérequis.

A	**Setup & Configuration**	9h	-	Yani
B	**Game Engine (serpent solo)**	14h	A	Yani
C	Rendu Canvas (serpent)	7h	A	Amine
D	Interface mobile (joystick)	6h	A	Malcolm
E	**Système d'orbes**	7h	B	Yani
F	**Multi-clients Socket.IO**	5h	B,C,D	Yani
G	**Détection collisions**	14h	F	Yani
H	**Élimination & mode spectateur**	4h	G	Yani
I	Responsive iOS/Android	4h	D,F	Malcolm
J	**Pièces & Coffres**	7h	H	Yani
K	**Bonus (Aimant/Invincib./Fantôme)**	8h	J	Yani
L	UI bonus mobile	3h	K,I	Malcolm
M	**Effets visuels & animations**	11h	K	Amine
N	Classement Top 5 + Kill Feed	5h	H,E	Amine
O	Écran de victoire	3h	H	Amine
P	**Tests charge & optimisation**	8h	M,N,L	Tous
Q	**Documentation & Soutenance**	18h	P,O	Tous

Chemin critique (en rouge) : A → B → F → G → H → J → K → M → P → O
Durée totale du chemin critique : ~98h de travail effectif sur 6 semaines
Ces tâches ne peuvent pas être retardées sans impacter la date de livraison finale.

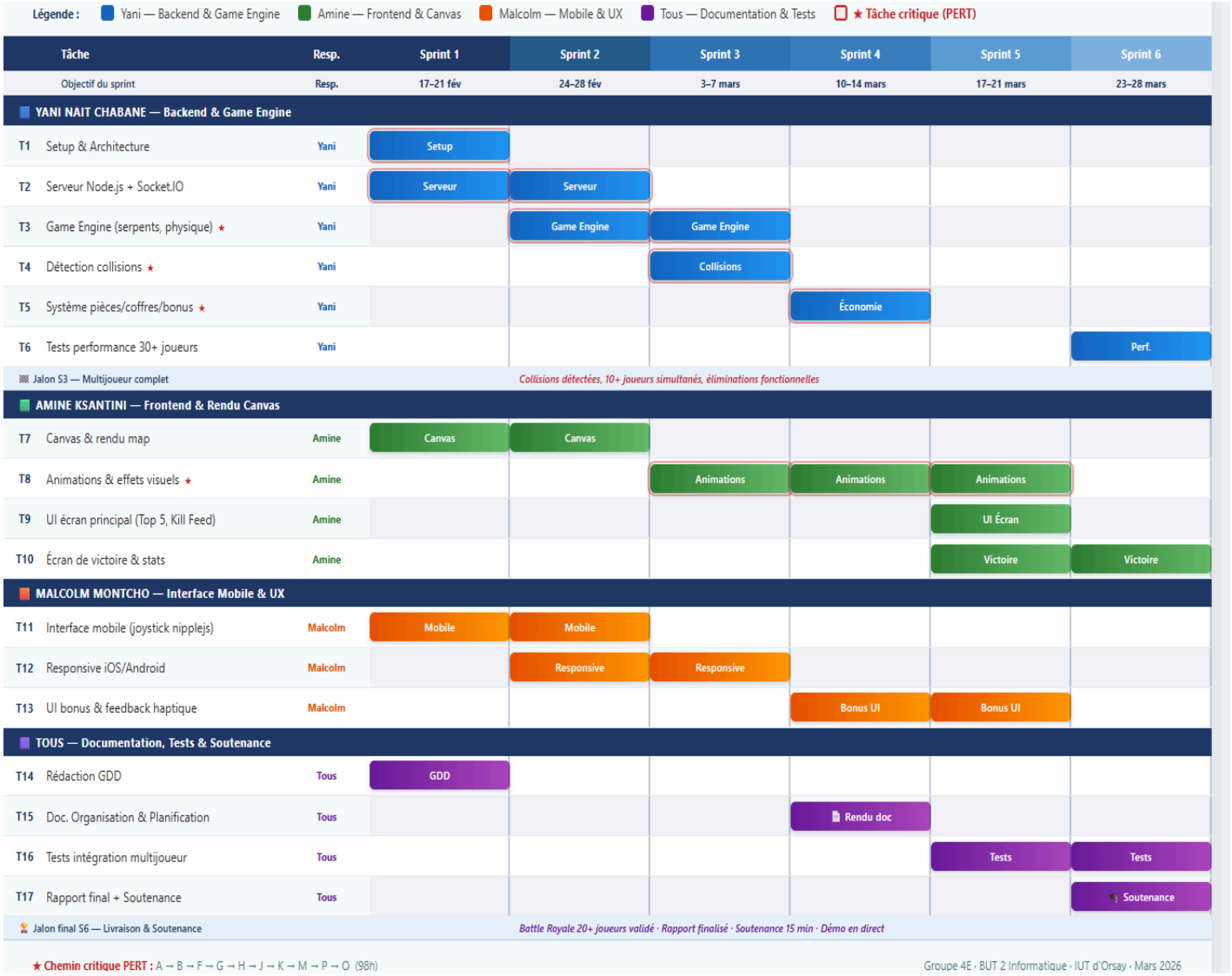
3.1 Représentation du graphe PERT

Le graphe PERT suit la logique de dépendances suivante (lecture de gauche à droite) :



4. Diagramme de GANTT

Le diagramme de GANTT ci-dessous est dérivé du PERT. Il représente le planning d'exécution des tâches sur les 6 semaines du projet. Chaque tâche est affichée avec sa durée (cases colorées) selon le membre responsable.



4.1 Jalons du projet

Semaine	Jalon	Livrable
S1	**Setup**	Serveur Node.js fonctionnel, connexion Socket.IO établie, map Canvas visible
S2	**Prototype solo**	1 serpent contrôlable depuis mobile, collecte d'orbes, croissance visible
S3	**Multijoueur**	10+ joueurs connectés simultanément, collisions détectées, éliminations fonctionnelles
S4	**Économie de jeu**	Pièces, coffres (3 niveaux) et les 3 bonus (Aimant, Invincibilité, Fantôme) opérationnels
S5	**Interface complète**	Top 5 temps réel, Kill Feed, effets visuels, interface mobile responsive et retours haptiques
S6	**Finalisation**	Battle Royale 20+ joueurs validé, documentation finalisée, soutenance préparée

5. Répartition des Tâches

La répartition des tâches est définie selon les compétences et appétences de chaque membre de l'équipe, tout en assurant une charge de travail équilibrée. Des zones de collaboration communes sont identifiées pour les tâches d'intégration.

5.1 Yani NAIT CHABANE – Backend & Game Engine

Domaine principal : Développement serveur, logique de jeu, performance

- Architecture serveur Node.js + Express.js + Socket.IO
- Game Engine : boucle de jeu à 60 ticks/seconde, gestion de l'état global
- Logique des serpents : déplacement, croissance, calcul de positions
- Système de détection de collisions (corps, tête-à-tête, bordures)
- Système d'élimination, gestion du mode spectateur
- Économie : pièces, coffres (Bronze/Argent/Or), logique des 3 bonus
- Optimisation pour 30+ joueurs simultanés, tests de charge

Charge estimée : ~76h | Formation prévue : Socket.IO avancé, optimisation Node.js

5.2 Amine KSANTINI – Frontend & Rendu Canvas

Domaine principal : Rendu graphique, animations, interface projecteur

- Rendu HTML5 Canvas : map rectangulaire, grille hexagonale, serpents
- Animations des serpents : effets néon, traînées lumineuses, glow pulsant
- Effets visuels de mort : explosion, particules, flash
- Auras des bonus (vert Aimant, bleu Invincibilité, violet Fantôme)
- Interface écran principal : classement Top 5 temps réel, Kill Feed
- Écran de victoire : podium, statistiques de partie
- Countdown de démarrage et transitions d'écrans

Charge estimée : ~62h | Formation prévue : HTML5 Canvas 2D avancé, optimisation 60 FPS

5.3 Malcolm MONTCHO – Interface Mobile & UX

Domaine principal : Interface mobile, expérience joueur, accessibilité

- Interface manette mobile : joystick virtuel (nipplejs), ergonomie tactile
- Connexion et envoi de commandes au serveur (player:move à 60/sec)
- Affichage des stats personnelles : score, rang, pièces, bonus en cours
- Responsive design : compatibilité iOS (Safari) et Android (Chrome)
- Bouton d'activation des bonus avec timer circulaire et barre de progression
- Feedback haptique : vibrations selon événements (collecte, mort, coffre)

- Page de connexion : saisie pseudo + choix d'émoji de serpen

Charge estimée : ~44h | Formation prévue : nipplejs, Vibration API, CSS responsive

5.4 Tâches communes (Tous)

- Rédaction du Game Design Document (GDD) – Semaine 1
- Rédaction du document Organisation & Planification – Semaine 6
- Tests d'intégration multijoueur (simulation 20-30 joueurs) – Semaine 5-6
- Débogage et résolution de bugs transversaux – Semaine 6
- Rapport final technique – Semaine 6
- Préparation et répétition de la soutenance (15 min) – Semaine 6

6. Organisation de l'Équipe

6.1 Outils de Communication

L'équipe utilise les outils de communication suivants pour coordonner le travail au quotidien :

Discord	Communication quotidienne	Serveur dédié au projet avec channels séparés : #général, #backend, #frontend, #mobile, #bugs. Réunions vocales hebdomadaires le lundi.
WhatsApp	Urgences & coordination	Groupe de messages pour les alertes rapides, les blocages critiques ou la coordination en séance.
GitHub Issues	Suivi des tâches	Chaque tâche est tracée comme une issue GitHub avec labels (bug, feature, doc), assignation et milestone par semaine.
GitHub Projects	Kanban board	Tableau Kanban (To Do / In Progress / Review / Done) mis à jour en temps réel par chaque membre.
Réunions en présentiel	Synchronisation	Point hebdomadaire en séance encadrée + réunions ad hoc en cas de besoin. Compte-rendu déposé sur GitHub.

6.2 Versioning et Gestion du Code

La gestion du code source est assurée via Git avec hébergement sur GitHub. Un workflow de branches structuré garantit la stabilité du code principal.

Plateforme	GitHub – dépôt privé partagé entre les 3 membres et accessible à l'encadrant
Branche principale	main : code stable et testé uniquement. Toute modification passe par une Pull Request revue par un autre membre.
Branche de développement	develop : branche d'intégration continue. Les branches de fonctionnalités y sont mergées.
Branches feature	feature/nom-fonctionnalite (ex: feature/collision-system, feature/mobile-joystick). Une branche par tâche.

Convention de commits	Format conventionnel : feat: / fix: / docs: / refactor: / test: suivi d'une description courte en français.
Code reviews	Toute Pull Request doit être approuvée par au moins un autre membre avant le merge. Délai de réponse : 24h maximum.
Releases	Un tag Git est créé à chaque fin de semaine (v1.0-S1, v2.0-S2...) pour marquer les jalons du projet.

6.3 Environnements de Développement

IDE	Visual Studio Code avec extensions : ESLint, Prettier, GitLens, Live Share (collaboration en temps réel)
Runtime	Node.js 20.x LTS – même version pour tous les membres (fichier .nvmrc dans le dépôt)
Formatage	Prettier configuré (.prettierrc) pour uniformiser le style de code : 2 espaces, simple quotes, no trailing commas
Linting	ESLint avec règles communes pour détecter les erreurs et maintenir la qualité du code
Test réseau	Partage de connexion / hotspot pour simuler le WiFi local lors des tests de développement individuels

6.4 Méthode de Travail

L'équipe adopte une approche agile légère avec des sprints hebdomadaires alignés sur les 6 semaines de projet :

Cycle hebdomadaire (Sprint)

1. Lundi : Réunion de planification du sprint (30 min) – définition des tâches de la semaine et assignation
2. Mardi-Jeudi : Développement individuel selon les tâches assignées, communication via Discord
3. Vendredi : Merge des branches, intégration, tests collectifs et mise à jour du Kanban
4. Fin de semaine : Rétrospective courte (ce qui a bien marché, les blocages, les ajustements)

6.5 Gestion des Risques

Latence trop élevée avec 30 joueurs	Moyen	Optimisation dès S3 : compression des messages, envoi différentiel, réduction du payload Socket.IO
Incompatibilité navigateur mobile	Faible	Tests sur iOS Safari et Android Chrome dès S2. Utilisation de polyfills si nécessaire.
Dérive du planning	Moyen	Revue hebdomadaire du GANTT, priorisation des fonctionnalités essentielles (MVP), fonctionnalités bonus clairement identifiées

Conflits de merge Git	Moyen	Branches courtes et fréquents rebases sur develop. Chaque membre travaille sur des fichiers distincts autant que possible.
Manque de WiFi performant	Faible	Tests avec hotspot personnel dès S3. Prévision d'un routeur dédié pour la soutenance si nécessaire.

7. Conclusion

Ce document de planification définit un cadre de travail structuré et réaliste pour le développement d'Emoji Slither en 6 semaines. La répartition des tâches joue sur les forces de chaque membre : Yani sur le backend et la logique de jeu, Amine sur le rendu graphique et les animations, Malcolm sur l'expérience utilisateur mobile.

Le diagramme de PERT identifie un chemin critique centré sur le game engine et le système de collisions, qui constituent le cœur technique du projet. Le GANTT assure une progression continue et des livrables hebdomadaires vérifiables.

Les outils choisis (GitHub, Discord, VS Code) sont des standards de l'industrie qui permettront à l'équipe de travailler efficacement, de manière collaborative et transparente. La méthode agile légère adoptée garantit une capacité d'adaptation face aux imprévus tout en maintenant le cap sur les objectifs.

Que le meilleur serpent gagne !

Yani NAIT CHABANE • Amine KSANTINI • Malcolm MONTCHO